

FULL STACK DEVELOPMENT

FASE 2 | AULA 02 -

REST NA PRÁTICA COM NODEJS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMAND

O QUE VEM POR AÍ?

Nesta aula, aprenderemos o REST na prática, implementando uma API RESTful usando Node.js e Express. Vamos explorar os conceitos teóricos e aplicá-los em um exemplo prático. Ao final da aula, você será capaz de criar uma API que segue os princípios RESTful, utilizando métodos HTTP e identificadores de recursos claros.



HANDS ON

Nesta aula prática, criamos duas APIs para reforçar nosso conhecimento no desenvolvimento de APIs utilizando o estilo arquitetural REST. Esta abordagem nos permitiu aplicar os conceitos teóricos na prática, consolidando nosso entendimento sobre a implementação de APIs RESTful.



SAIBA MAIS

REST, ou Representational State Transfer, é um estilo arquitetural para a construção de serviços web escaláveis. Introduzido por Roy Fielding em sua tese de doutorado em 2000, REST se baseia em um conjunto de restrições arquiteturais que, quando aplicadas como um todo, enfatizam a escalabilidade, simplicidade, desempenho e facilidade de manutenção.

Princípios do REST

1. Client-Server: a arquitetura REST é baseada na separação de responsabilidades entre o cliente e o servidor. Isso significa que o cliente é responsável por gerenciar a interface do usuário e o estado da aplicação, enquanto o servidor é responsável por gerenciar os dados e a lógica do negócio. Essa separação permite que cada componente evolua de forma independente.

2. Stateless: no REST, cada solicitação do cliente para o servidor deve conter todas as informações necessárias para entender e processar a solicitação. Isso significa que o servidor não armazena nenhum estado da sessão do cliente. A ausência de estado (statelessness) permite que o servidor trate cada solicitação como uma operação independente, facilitando a escalabilidade.

3. Cacheable: as respostas do servidor devem ser explicitamente rotuladas como cacheáveis ou não cacheáveis. Se uma resposta for cacheável, o cliente pode reutilizá-la para solicitações futuras, reduzindo a carga no servidor e melhorando a performance da aplicação.

4. Uniform Interface: a interface uniforme é uma das características mais importantes do REST. Ela padroniza a maneira como os clientes e servidores se comunicam, facilitando a interoperabilidade entre diferentes sistemas. Uma interface uniforme deve seguir os seguintes princípios:

- Identificação de recursos: cada recurso é identificado por uma URI (Uniform Resource Identifier) única.
- Manipulação de recursos através de representações: os clientes interagem com os recursos utilizando suas representações (por exemplo: JSON e XML).
- Mensagens autodescritivas: cada mensagem contém informações suficientes para descrever como processá-la.

- Hypermedia as the Engine of Application State (HATEOAS): os clientes interagem com a aplicação inteiramente através de hypermedia fornecida dinamicamente pelo aplicativo.

5. Layered System: um sistema em camadas é projetado para melhorar a escalabilidade e modularidade. Cada camada na arquitetura tem uma função específica e pode ser desenvolvida e modificada de forma independente. Por exemplo, uma camada pode ser responsável pelo roteamento de solicitações, enquanto outra pode gerenciar a lógica de negócios.

Operações HTTP no REST

O REST utiliza os verbos HTTP para realizar operações nos recursos. Os mais comuns são:

- GET: recupera uma representação de um recurso.
- POST: envia dados para o servidor para criar um novo recurso.
- PUT: atualiza um recurso existente ou cria um novo recurso se ele não existir.
- DELETE: remove um recurso.
- PATCH: aplica modificações parciais a um recurso.

Agora que sabemos o que é o REST, você saberia o que é o famoso RESTful ?

Caso a sua resposta seja não: o RESTful refere-se à implementação prática do estilo arquitetural REST (Representational State Transfer). Quando dizemos que uma API é RESTful, estamos dizendo que ela segue os princípios e restrições do REST para criar uma interface web que é escalável, eficiente e fácil de usar. APIs RESTful são projetadas para serem simples e utilizam os métodos HTTP (GET, POST, PUT, DELETE, etc.) para realizar operações em recursos identificados por URIs (Uniform Resource Identifiers).

Vejamos quais seriam as diferenças entre REST e RESTful para que você possa dominar este assunto.

REST

- Objetivo: o objetivo do REST é fornecer um guia para a construção de sistemas escaláveis e eficientes na web.

RESTful

- Implementação: RESTful refere-se à aplicação prática dos princípios do REST em um serviço web.
- API: quando dizemos que uma API é RESTful, significa que ela segue os princípios do REST, utilizando métodos HTTP e URIs para operar em recursos.
- Prática: uma API RESTful deve ser stateless, cacheable, e deve utilizar uma interface uniforme para interagir com os recursos.

Em resumo, enquanto REST é a teoria e o conjunto de princípios arquiteturais, RESTful é a aplicação prática desses princípios em APIs e serviços web. Uma API RESTful é aquela que implementa os princípios do REST para fornecer uma interface de comunicação eficiente e escalável entre sistemas na web.

Para esclarecermos mais o conceito de RESTful, criaremos uma API simples para gerenciar recursos de usuários utilizando Node.js e Express. Este exemplo prático mostrará como implementar os princípios do REST em uma API real.

Obs.: não se preocupe em entender o que seria o Express e o body-parser nesse momento, nós teremos uma disciplina dedicada a desenvolvimento de API com eles.

Primeiro, crie uma nova pasta para o seu projeto e inicialize um novo projeto Node.js:

```
mkdir user-api  
cd user-api  
npm init -y
```

Instale as dependências necessárias: Express para criar o servidor web e body-parser para lidar com os dados das requisições HTTP.

```
npm install express body-parser
```

A estrutura básica do projeto será assim:

```
user-api/  
├── models/  
│   └── user.js  
├── routes/  
│   └── users.js  
├── index.js  
└── package.json
```

Crie o arquivo models/user.js para definir o modelo User:

```
class User {  
  constructor(id, name, email) {  
    this.id = id;  
    this.name = name;  
    this.email = email;  
  }  
}  
  
module.exports = User;
```

Crie o arquivo routes/users.js para definir as rotas da API:

```
const express = require('express');  
const router = express.Router();  
const User = require('../models/user');  
  
let users = [  
  new User(1, 'Thiago S Adriano', 'tadriano@fiap.com.br')  
];  
  
// GET: /api/users  
router.get('/', (req, res) => {  
  res.json(users);  
});  
  
// GET: /api/users/:id  
router.get('/:id', (req, res) => {  
  const user = users.find(u => u.id === parseInt(req.params.id));  
  if (!user) return res.status(404).send('User not found');  
  res.json(user);  
});  
  
// POST: /api/users  
router.post('/', (req, res) => {
```

```
const { name, email } = req.body;
const user = new User(users.length + 1, name, email);
users.push(user);
res.status(201).json(user);
});

// PUT: /api/users/:id
router.put('/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).send('User not found');

  const { name, email } = req.body;
  user.name = name;
  user.email = email;
  res.json(user);
});

// DELETE: /api/users/:id
router.delete('/:id', (req, res) => {
  const userIndex = users.findIndex(u => u.id === parseInt(req.params.id));
  if (userIndex === -1) return res.status(404).send('User not found');

  users.splice(userIndex, 1);
  res.status(204).send();
});

module.exports = router;
```

Crie o arquivo index.js para configurar o servidor Express e usar as rotas:

```
const express = require('express');
const bodyParser = require('body-parser');
const users = require('./routes/users');

const app = express();
const port = 3000;

app.use(bodyParser.json());
app.use('/api/users', users);

app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

Agora execute o comando `node index.js` e abra no seu navegador o seguinte endereço: <http://localhost:3000/api/users>

Caso todos os passos anteriores tenham sido criados corretamente, você terá o seguinte resultado:



```
{
  {
    "id": 1,
    "name": "Thiago S Adriano",
    "email": "tadriano@fiap.com.br"
  }
}
```

Figura 1 - Retorno de API RestFUL
Fonte elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre o REST e RESTful e implementamos uma API RESTful usando Node.js e Express, depois que aprendemos sobre os princípios do REST e operações HTTP.

EMANDA

REFERÊNCIAS

Introdução a web services RESTful. **devmedia**. 2023. Disponível em: <https://www.devmedia.com.br/introducao-a-web-services-restful/37387#:~:text=implanta%C3%A7%C3%A3o%20dos%20servi%C3%A7os%20RESTful,entidade%20em%20um%20dado%20momento.> Acesso em: 09 ago. 2024.

Padrão REST. **brunobrito**. 2020. Disponível em: <https://www.brunobrito.net.br/padrao-rest/>. Acesso em: 09 ago. 2024.

O que é uma API REST? **redhat**. 2023. Disponível em: <https://www.redhat.com/pt-br/topics/api/what-is-a-rest-api>. Acesso em: 09 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: REST, RESTful, HTTP.

EMSE